Деплой RTK CRM

Приложение не привязано к конкретному облаку: СУБД, объектное хранилище, кэш и список разрешённых origin задаются переменными окружения. Одна и та же сборка разворачивается в обоих поддерживаемых контурах.

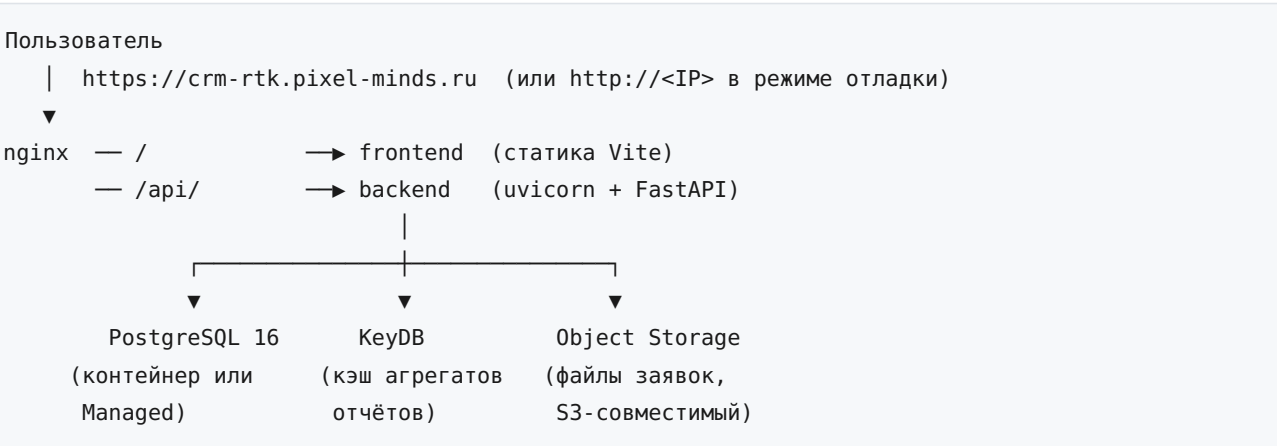
Контур	Состав	Назначение
1. Yandex Cloud (основной)	BM Compute + nginx + PostgreSQL + Object Storage + KeyDB	Целевой контур: данные в РФ, 152-ФЗ, приказ ФСТЭК № 117
2. Render + Vercel + Supabase (альтернативный)	Render (backend), Vercel (frontend), Supabase (PostgreSQL)	Быстрый публичный стенд без своей инфраструктуры
3. Локально	Docker Compose или SQLite + Vite	Разработка и тесты

- 1. Yandex Cloud
- 2. Render + Vercel + Supabase
- 3. Локальный запуск
- 4. Общее для всех контуров

---

1. Yandex Cloud (основной контур)

Всё разворачивается на одной BM: PostgreSQL, KeyDB, backend (FastAPI), frontend (React/Vite) и nginx. Пошаговая инструкция по созданию BM — в `infra/yandex-cloud/README.md`.



Все контейнеры в одной docker-сети, наружу смотрят только nginx (80/443). Backend, база и кэш снаружи недоступны.

1.1. Рекомендуемые ресурсы ВМ

Параметр	Значение
vCPU	2 vCPU, **100 %** уровня производительности
RAM	4 ГБ
Диск	30 ГБ
IP	статический (нужен для A-записи и сертификата)
Образ	Ubuntu 22.04 LTS

Замеры при сборке образов: пик памяти `npm ci && tsc && vite build` — 1.69 ГБ, `pip install` бэкенда — 0.23 ГБ, рантайм всего стека — 143 МиБ в покое и ~210 МиБ под нагрузкой 50 пользователей. 4 ГБ хватает с запасом.

Доля vCPU важна отдельно: нагрузочный профиль из `tests/load/` упирается примерно в 1.6 ядра (backend ~105 %, PostgreSQL ~57 %). На 20 %-й доле этот профиль не покажет проектных цифр — для демонстрации переключайте на 100 % (меняется без пересоздания ВМ).

1.2. Установка на ВМ

```
# на ВМ, после клонирования репозитория
cd infra/yandex-cloud
sudo ./setup-vm.sh
```

Скрипт ставит Docker, certbot, настраивает UFW и swap, копирует `.env.prod.example` в `.env`. Для приватного репозитория нужен `GITHUB_TOKEN` или deploy-key.

Далее заполните `.env` (шаблон — `infra/yandex-cloud/.env.prod.example`):

```
DOMAIN=rtk-crm.ru
TLS_ENABLED=true

# Вариант А: Managed PostgreSQL
# DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@c-<cluster-
id>.rw.mdb.yandexcloud.net:6432/rtk_crm?ssl=require
# Вариант Б: PostgreSQL в контейнере на ВМ
DATABASE_URL=postgresql+asyncpg://rtk_user:<password>@postgres:5432/rtk_crm

# Кэш агрегатов отчётов
REDIS_URL=redis://keydb:6379/0

# Файлы заявок: Object Storage
S3_ENDPOINT=storage.yandexcloud.net
S3_SECURE=true
S3_REGION=ru-central1
S3_ACCESS_KEY=<yandex-access-key>
S3_SECRET_KEY=<yandex-secret-key>
S3_BUCKET=rtk-crm-files

SECRET_KEY=<уникальная строка не менее 32 символов>
CORS_ORIGINS=https://rtk-crm.ru
SEED_DEMO_DATA=true

# Сводка по карточке (ФТ-6). Без ключа функция выключена: 503 в API и
# пояснение во вкладке «Сводка». Сертификат НУЦ Минцифры вшит в образ,
# задавать GIGACHAT_CA_BUNDLE не нужно.
GIGACHAT_CREDENTIALS=<authorization-key-base64>
GIGACHAT_SCOPE=GIGACHAT_API_B2B
GIGACHAT_MODEL=GigaChat
GIGACHAT_FALLBACK_ENABLED=true
```

Имена переменных должны совпадать с теми, что читает `backend/core/config.py` (`SECRET_KEY`, `SEED_DEMO_DATA`, `CORS_ORIGINS`, `S3_*`). `ENVIRONMENT=production` включает проверку длины `SECRET_KEY` при старте — со слабым ключом приложение не поднимется.

Запуск:

```
docker compose -f infra/yandex-cloud/docker-compose.prod.yml --env-file .env up -d
```

1.3. Два режима nginx

Режим выбирается переменной `TLS_ENABLED`:

TLS_ENABLED	Что поднимается	Когда нужен
`false`	только 80, отвечает по IP	отладка, домена и сертификата ещё нет
`true`	80 (редирект) + 443 с сертификатом	демонстрация жюри

В режиме `true` сертификат нужно выпустить **до** старта nginx: `certbot --standalone` занимает порт 80, поэтому одновременно с работающим nginx он не отработает. Сертификат можно получить и через Yandex Certificate Manager, если домен делегирован в Yandex Cloud DNS.

1.4. Managed-сервисы (опционально)

Вместо контейнеров на ВМ можно подключить управляемые сервисы — код менять не нужно, достаточно переменных:

Сервис	Переменная	Значение
Yandex Managed PostgreSQL	`DATABASE_URL`	хост вида `c-<id>.rw.mdb.yandexcloud.net:6432`, `?ssl=require`
Yandex Object Storage	`S3_ENDPOINT`, `S3_SECURE`, `S3_REGION`	`storage.yandexcloud.net`, `true`, `ru-central1`
Yandex Managed Redis	`REDIS_URL`	адрес кластера с TLS

Managed PostgreSQL требует `NullPool` (уже настроен в `backend/db/session.py`): внешний пулер в режиме transaction pooling не должен удерживать соединения приложения.

1.5. Бэкапы

`infra/yandex-cloud/backup.sh` кладётся в `/etc/cron.daily/rtk-backup` скриптом `setup-vm.sh` и делает `pg_dump` раз в сутки.

---

2. Render + Vercel + Supabase (альтернативный контур)

Контур без своей инфраструктуры: frontend — статика на Vercel, backend — контейнер на Render, база — Supabase PostgreSQL.

```
Vercel (frontend, статика Vite)
   │  https://<project>.vercel.app
   ▼
Render (backend, Docker: uvicorn + FastAPI)
   │  https://<service>.onrender.com
   ▼
Supabase PostgreSQL (Session Pooler, порт 5432)
```

Отличие от контура на ВМ: frontend и backend живут на разных доменах, поэтому запросы идут cross-origin и `CORS_ORIGINS` обязателен, а `VITE_API_URL` указывает на абсолютный адрес backend.

2.1. Supabase (база данных)

Connection string (Session Pooler, порт 5432):

```
DATABASE_URL=postgresql+asyncpg://<user>:<password>@<host>.pooler.supabase.com:5432/postgres
```

Требования:

- префикс `postgresql+asyncpg://` (не `postgresql://`);
- порт `5432` (Session Pooler — держит постоянное соединение, нужно asyncpg);
- SSL включён по умолчанию, дополнительных параметров не требуется.

> Пароли в репозиторий не коммитятся. Значения — только в Render → Environment и локальном `backend/.env`.

2.2. Render (backend)

Тип: **Web Service** (Docker).

В репозитории есть готовый Blueprint `render.yaml`: он задаёт сервис, регион, путь к Dockerfile и переменные окружения, поэтому в панели Render достаточно подключить репозиторий и заполнить секреты (`sync: false`). Если создавать сервис вручную, параметры те же:

Параметр	Значение
Repository	`Pavel1778/rtk-crm`
Branch	`main`
Environment	**Docker**
Dockerfile Path	`backend/Dockerfile`
Docker Context Directory	`backend`
Region	**Frankfurt (EU Central)** — ближе к Supabase eu-west-1
Instance Type	Free

Переменные окружения (Render → Environment):

Ключ	Значение
`DATABASE_URL`	значение из Supabase
`SECRET_KEY`	уникальная строка не менее 32 символов
`ENVIRONMENT`	`production`; включает HSTS и строгую проверку `SECRET_KEY`
`CORS_ORIGINS`	`https://<project>.vercel.app,http://localhost:5173`
`CORS_ORIGIN_REGEX`	regex для хостов предпросмотра (необязательно)
`SEED_DEMO_DATA`	`true`
`MOCK_MODE`	`true`
`GIGACHAT_CREDENTIALS`	ключ авторизации из личного кабинета GigaChat Developers
`GIGACHAT_SCOPE`	`GIGACHAT_API_B2B`
`GIGACHAT_MODEL`	`GigaChat`
`GIGACHAT_FALLBACK_ENABLED`	`true` — демо-сводка вместо 502 при сбое сервиса

> Сертификат НУЦ Минцифры вшит в образ (`backend/Dockerfile`) и попадает в `/certs`, поэтому на Render дополнительно настраивать TLS не нужно: `GIGACHAT_CA_BUNDLE` уже указывает на этот путь по умолчанию. Если `GIGACHAT_CREDENTIALS` не задан, вкладка «Сводка» в карточке сообщает, что сервис не настроен (ответ 503).
>
> `ENVIRONMENT` должен быть ровно `production` (или `prod`). При другом значении HSTS не отдаётся, а проверка длины `SECRET_KEY` не применяется.

`backend/start.sh` — вспомогательный скрипт для деплоя без Docker (задаётся как Start Command); в образ он не встроен. Выполняет те же шаги: `create_tables()` → `ensure_schema()` → `seed_reference()`.

2.3. Vercel (frontend)

Параметр	Значение
Framework Preset	Vite
Root Directory	`frontend`
Build Command	`npm run build`
Output Directory	`dist`

Ключ	Значение
`VITE_API_URL`	`https://<service>.onrender.com`

Frontend — SPA на React Router, поэтому нужен rewrite всех путей на `index.html` (иначе 404 на `/reports`, `/directories`, `/settings`). Файл `frontend/vercel.json` уже в репозитории:

```
{ "rewrites": [{ "source": "/(.*)", "destination": "/index.html" }] }
```

2.4. Проверка контура

```
GET https://<service>.onrender.com/           -> {"service":"RTK CRM",...}
GET https://<service>.onrender.com/health     -> {"status":"ok"}
GET https://<service>.onrender.com/api/health -> {"status":"ok","app":"RTK CRM","database":"ok"}
GET https://<service>.onrender.com/healthz    -> {"status":"ok"}
GET https://<service>.onrender.com/readyz     -> {"status":"ok","checks":{...}}
GET https://<service>.onrender.com/metrics    -> метрики Prometheus
GET https://<service>.onrender.com/docs       -> Swagger UI
```

2.5. Типичные ошибки

Симптом	Причина	Решение
Build: `backend/ $` / ошибка pre-deploy	В Render заполнено `Pre-Deploy Command`	Очистить поле
`relation "users" does not exist`	Схема не создана	Проверить, что старт шёл через `start.sh`/CMD
`connection timeout` к Supabase	Порт `6543` (transaction pooler) с asyncpg	Использовать `5432` (session pooler)
CORS error в браузере	`CORS_ORIGINS` без домена Vercel	Строка через запятую без пробе-лов
`Invalid (interleaved) UTF-8` в логах	Нет `PYTHONIOENCODING`	Уже задано в Dockerfile
Медленные ответы	Render в US, Supabase в EU	Region Render: Frankfurt

3. Локальный запуск

3.1. Через Docker Compose

```
docker compose up -d --build
# frontend: http://localhost:5173
# backend:  http://localhost:8000/docs
```

3.2. Без Docker

```
# Backend (SQLite, внешняя СУБД не нужна)
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r backend/requirements.txt
# .env в корне репозитория: DATABASE_URL=sqlite+aiosqlite:///./dev.db
python scripts/run_dev.py

# Frontend
cd frontend
npm install
npm run dev
```

---

4. Общее для всех контуров

4.1. Как это работает при старте

1. `Dockerfile` ставит зависимости из `backend/requirements.txt` и запускает `uvicorn app.main:app --host 0.0.0.0 --port 8000` (код в `/app`, `PYTHONPATH=/`).
2. При старте (`app.main:lifespan`) создаётся схема БД (`create_tables()`) и **сверяется с моделями** (`ensure_schema()`: добавляет недостающие колонки и индексы, заменяет устаревшие глобальные UNIQUE на составные по `scope`), затем идемпотентно загружаются справочники (`seed_reference()`: 14 этапов B2B + 4 этапа B2C, пользователи, направления, продукты).
3. Если `SEED_DEMO_DATA=true`, при старте дополнительно заполняются демо-вузы и взаимодействия (только для пустой базы).

> Сверка схемы нужна, потому что `create_all(checkfirst=True)` существующие таблицы не изменяет: база, созданная ранней версией приложения, молча оставалась без `scope`, и загрузка демо-данных падала с `column workflow_stages.scope does not exist`. `ensure_schema()` доводит такую схему до моделей, не удаляя данные, и отмечает головную ревизию Alembic, чтобы последующий `alembic upgrade head` не пытался создать уже существующие таблицы.

4.2. Режимы аутентификации

Режим	`MOCK_MODE`	Что нужно	Использование
JWT + БД	`true`	Только БД и `SECRET_KEY`	Хакатонный контур, оба облака
Keycloak	`false`	Keycloak, realm, клиент и адаптер токенов	Закрытый контур заказчика

Сейчас API реализует JWT-аутентификацию через `POST /api/auth/login`. Значение `false` зарезервировано под Keycloak и не переключает backend автоматически.

4.3. Демо-доступ

Роль	Email	Пароль
Администратор	`admin@rtk.ru`	`admin123`
Руководитель	`manager@rtk.ru`	`manager123`
КАМ	`kam@rtk.ru`	`kam123`

4.4. Безопасность

- Пароли БД и `SECRET_KEY` — только в переменных окружения.
- `backend/.env` в `.gitignore`, в репозитории лишь `.env.example`.
- JWT-доступ ко всем эндпоинтам, кроме `/health` и логина.
- После публичного показа сменить пароль БД и отозвать использованные токены доступа.